

# WANDR

## AI Travel Planner

---

*Progress Report · May 2026*

---

**Stack:** LangGraph · Groq (llama-3.3-70b-versatile) · FastAPI · Vanilla JS

**Architecture:** Multi-Agent Supervisor Graph · 5 Specialised Worker Agents

**APIs:** REST Countries · Open-Meteo · Open Exchange Rates (all free, no key)

**Validation:** Deterministic per-agent output validators · Retry feedback loop

**Deployment:** FastAPI + SSE web app · CLI entry point · MemorySaver checkpointing

# Section 1 — System Architecture

## 1.1 Overview

Wandr is a multi-agent travel planning system built on LangGraph's StateGraph. A single natural-language user request flows sequentially through an input guardrail, a structured query parser, a human-in-the-loop confirmation screen, and then into a five-agent pipeline coordinated by a deterministic Supervisor node.

The system is designed around a core principle: every LLM call produces structured output (via Pydantic schemas and Groq's `with_structured_output`), and every agent output is validated by deterministic Python code before the workflow advances. This makes the system predictable, debuggable, and resilient to LLM output variability.

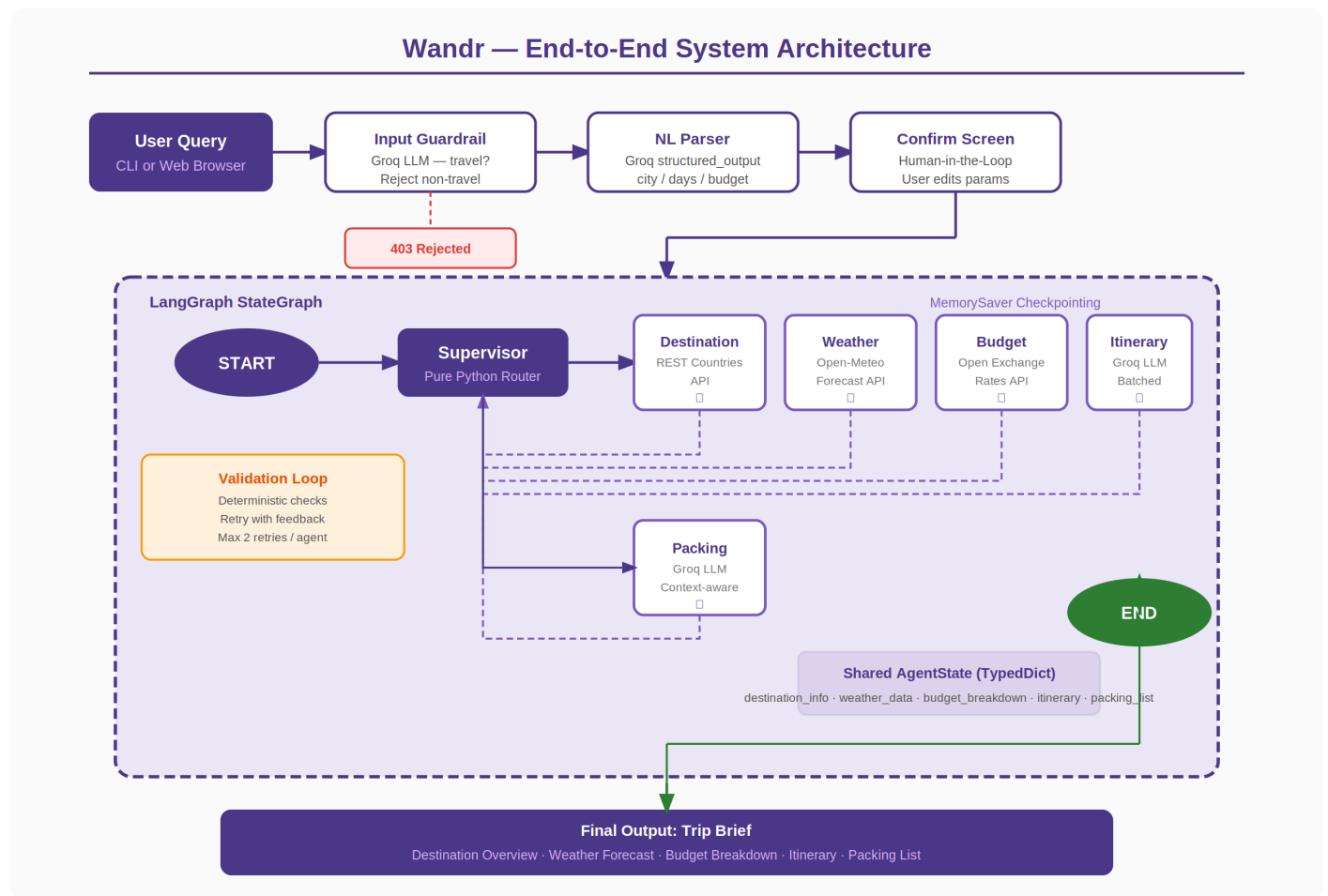


Figure 1 — End-to-end system architecture: from user query to trip brief

## 1.2 LangGraph StateGraph Topology

The graph is a directed StateGraph with one central Supervisor node and five Worker agent nodes. The topology is hub-and-spoke: every agent unconditionally returns to the Supervisor after completing, and the Supervisor uses conditional edges to decide what runs next. This keeps routing logic in one place and makes the execution trace easy to read.

| Agent             | Order | API / Tool                      | LLM Used                    | Output Field     |
|-------------------|-------|---------------------------------|-----------------------------|------------------|
| Destination Agent | ①     | REST Countries API              | Groq (overview paragraphs)  | destination_info |
| Weather Agent     | ②     | Open-Meteo Geocoding + Forecast | None — API only             | weather_data     |
| Budget Agent      | ③     | Open Exchange Rates API         | Groq (proportions only)     | budget_breakdown |
| Itinerary Agent   | ④     | None — LLM only                 | Groq (batched, 4 days/call) | itinerary        |
| Packing Agent     | ⑤     | None — LLM only                 | Groq (context-aware)        | packing_list     |

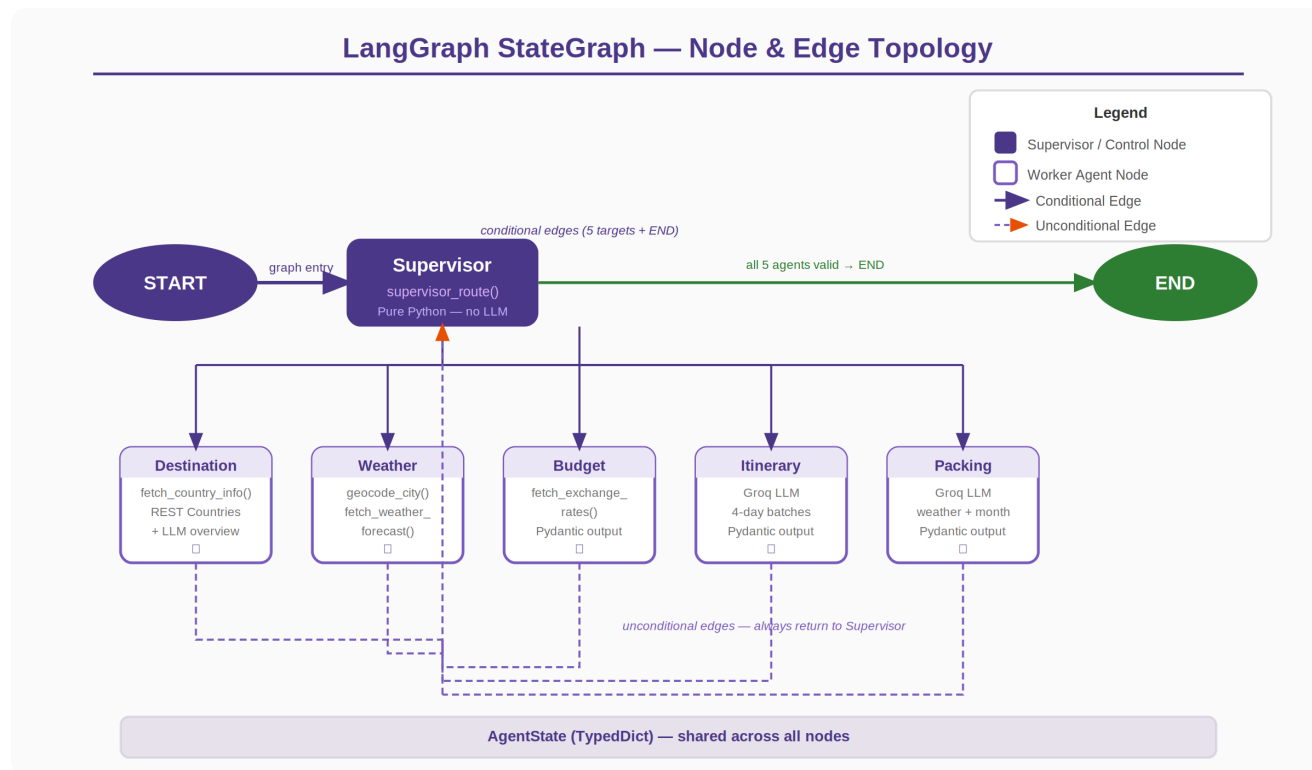


Figure 2 — LangGraph StateGraph: node and edge topology

### 1.3 Supervisor Node

The Supervisor is a pure Python routing function (`supervisor_route()` in `src/supervisor.py`) with no LLM calls. It reads `state["last_agent"]`, runs the corresponding output validator, and returns one of three outcomes:

- **Next agent name** — last agent passed validation, advance to the next pending agent
- **Same agent name** — output failed validation, retry with injected feedback (up to `MAX_RETRIES = 2`)
- **END** — all five agents have been validated (or accepted after hitting the retry cap)

Using deterministic routing instead of an LLM-based supervisor was a deliberate design choice. It makes routing fast, predictable, and fully logged. Every routing decision appears in `logs/supervisor.log` with the reason.

### 1.4 Validation Loop

After each agent runs, the Supervisor calls `validate_agent_output(agent_name, state)`. This function dispatches to a per-agent validator and runs deterministic checks only — no secondary LLM call is involved.

| Agent       | Validation Rules                                                                                                                                                  |
|-------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Destination | Required fields present (capital, currency, timezone); currency code is exactly 3 uppercase chars (ISO 4217)                                                      |
| Weather     | At least 5 forecast days present; all required fields per day; temperature within physical range (−60°C to 60°C)                                                  |
| Budget      | All 4 required categories present (accommodation, food, transport, activities); all <code>daily_amounts</code> positive; <code>duration_days</code> matches trip  |
| Itinerary   | Correct number of days; at least 4 segments per day; no generic location strings (downtown / the city / TBD / the area); all segments have non-empty descriptions |
| Packing     | Has both 'clothing' and 'documents' categories; no category has an empty item list                                                                                |

When validation fails and the retry count is below `MAX_RETRIES`, the supervisor writes a structured feedback string into `state["validation_feedback"][agent_name]`. On retry, the agent reads this string and prepends it to its prompt so the LLM knows exactly what to fix — for example: "Day 2 morning has a generic location: 'the city'". This targeted feedback means agents almost always self-correct on the first retry.

## 1.5 MemorySaver Checkpointing

MemorySaver is configured at graph compile time, keyed by `trip_id` (a UUID generated per request). Every node execution is checkpointed to RAM. This enables:

- **Crash recovery** — if the server process restarts mid-run, the graph can be resumed from the last checkpoint using the same `trip_id`
- **State retrieval** — the web server calls `travel_graph.get_state(config)` to retrieve the final state after streaming completes, which is how the `/api/plan/{id}/result` endpoint serves the trip brief

Production note: MemorySaver stores state in RAM — suitable for single-server deployments only. For multi-instance production, swap to a shared checkpointer (Redis or PostgreSQL via `langgraph-checkpoint-postgres`). The thread-ID-keyed interface is identical regardless of backend; the migration is a one-line change at graph compile time.

## 1.6 Why Parsing Runs Outside the Graph

Query parsing happens before the LangGraph graph starts. This implements the human-in-the-loop pattern correctly: the user sees the parsed parameters (city, days, budget, interests) on the Confirm screen and can edit them before any agent runs. If parsing were a node inside the graph, there would be no clean point to pause execution, surface the parsed data, accept user edits, and resume — LangGraph's streaming mode does not support interactive pauses mid-stream.

## Section 2 — API Tools

All four external data sources are completely free with zero registration and zero API key required. Every tool is decorated with LangChain's `@tool` decorator and called directly via `.invoke()` — not via LLM tool-use. This means tool execution is deterministic and never depends on the LLM deciding whether to call a tool.

### 2.1 Tool Overview

| API                  | Endpoint                               | Used By           | Returns                                                                     |
|----------------------|----------------------------------------|-------------------|-----------------------------------------------------------------------------|
| REST Countries v3.1  | restcountries.com/v3.1/name/{country}  | Destination Agent | Official name, capital, currencies, languages, timezone, region, flag emoji |
| Open-Meteo Geocoding | geocoding-api.open-meteo.com/v1/search | Weather Agent     | Latitude, longitude, timezone, country name for city name                   |
| Open-Meteo Forecast  | api.open-meteo.com/v1/forecast         | Weather Agent     | 7-day daily: max/min temp, precipitation sum, wind speed max                |
| Open Exchange Rates  | open.er-api.com/v6/latest/USD          | Budget Agent      | Live exchange rates for 166 currencies (USD base, free tier)                |

### 2.2 Tool Implementation Notes

#### REST Countries — `fetch_country_info()`

The API's capital field is a list (e.g. `["Tokyo"]`), not a string. The tool explicitly takes `capital[0]` to get the scalar form. On 404 or network error, the tool returns `{"error": "..."}` rather than throwing, so the validator catches it cleanly and triggers a retry.

#### Open-Meteo Geocoding — `geocode_city()`

For ambiguous city names, the API returns multiple results ranked by population. The tool always takes `results[0]`, which is the correct behaviour for travel planning — users mean the principal city when they say "Paris", not a smaller namesake.

#### Open-Meteo Forecast — `fetch_weather_forecast()`

The API returns parallel arrays (`dates[]`, `max_temps[]`, `min_temps[]`, etc.) rather than a list of day objects. The tool zips these arrays into a list of dicts (`{date, temp_max_c, temp_min_c, precipitation_mm, wind_max_kmh}`) before returning. This normalised format is easier for both the validator (index-based checks) and the packing agent (iterating over days).

#### Open Exchange Rates — `convert_currency()`

The free tier is locked to USD as the base currency, so direct cross-currency conversions are not available. All conversions use a two-leg USD pivot:  $\text{amount} \times (1 / \text{from\_rate}) \times \text{to\_rate}$ . The precision loss is negligible for budget estimates. The budget agent force-scales all LLM-generated category proportions to match the authoritative daily budget computed in Python — LLM arithmetic is never trusted for final money figures.

## Section 3 — Shared State and Data Model

All agents communicate exclusively through a shared AgentState TypedDict. No agent calls another agent directly. Each agent reads the fields it needs, writes only its own output field and last\_agent, and returns a partial dict that LangGraph merges into the full state.

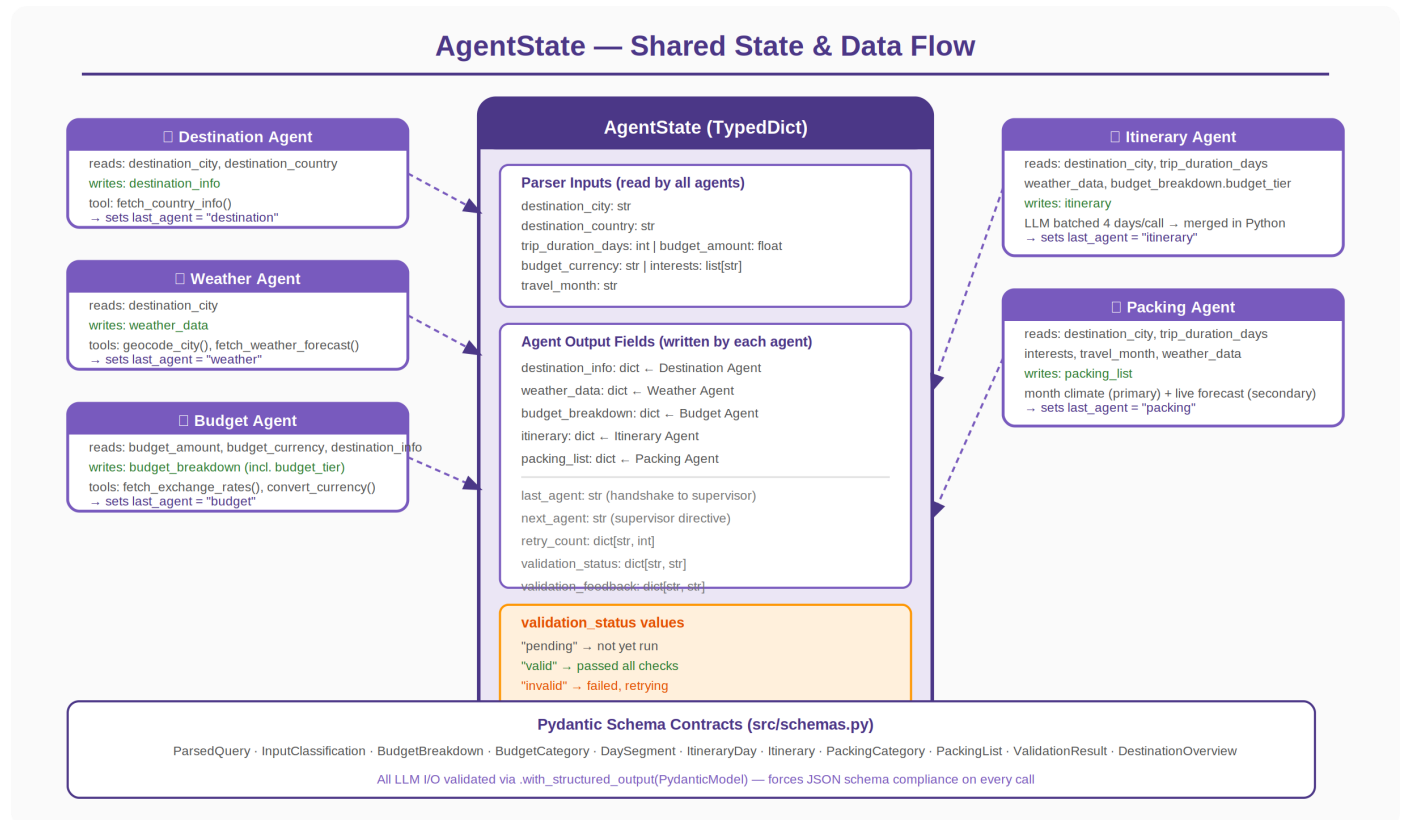


Figure 4 — AgentState: shared fields, per-agent reads/writes, and Pydantic schema contracts

### 3.1 Critical State Fields

| Field                      | Type        | Set By | Purpose                                          |
|----------------------------|-------------|--------|--------------------------------------------------|
| destination_city / country | str         | Parser | Primary inputs read by all agents                |
| trip_duration_days         | int         | Parser | Controls itinerary day count and budget division |
| budget_amount / currency   | float / str | Parser | Raw user budget for FX conversion                |
| travel_month               | str         | Parser | Drives packing list (not live forecast)          |



| Field                      | Type | Set By            | Purpose                                                                 |
|----------------------------|------|-------------------|-------------------------------------------------------------------------|
| destination_info           | dict | Destination Agent | Country metadata used by budget (currency) and itinerary (country name) |
| weather_data               | dict | Weather Agent     | 7-day forecast used by itinerary and packing agents                     |
| budget_breakdown           | dict | Budget Agent      | Includes budget_tier used by itinerary to calibrate venue style         |
| last_agent                 | str  | Each agent        | Handshake: tells supervisor which agent just completed                  |
| validation_status[agent]   | str  | Supervisor        | pending / valid / invalid / invalid_accepted                            |
| validation_feedback[agent] | str  | Supervisor        | Injected into agent prompt on retry — specific issues to fix            |

## 3.2 Budget Tier System

The Budget Agent classifies the user's budget into one of five tiers based on USD-equivalent daily spend. The Itinerary Agent reads this tier and selects a tier-specific style guide that controls the vocabulary and specificity of recommendations.

| Tier        | USD/day         | Accommodation         | Food                       | Transport         |
|-------------|-----------------|-----------------------|----------------------------|-------------------|
| budget      | < \$50          | Hostel dorms          | Street food stalls         | Metro / bus only  |
| mid-range   | \$50 – \$150    | 3-star hotels         | Casual restaurants         | Mix public + taxi |
| comfortable | \$150 – \$400   | 4-star hotels         | Quality restaurants        | Taxis + transfers |
| upscale     | \$400 – \$1,000 | 5-star hotels         | Fine dining, wine pairings | Private car hire  |
| luxury      | > \$1,000       | Palace hotels, villas | Michelin-starred only      | Helicopter, yacht |

## Section 4 — Web Layer Architecture

---

The web layer is a FastAPI application with four HTML screens, four API endpoints, a Server-Sent Events (SSE) streaming endpoint, and a thread pool that bridges the synchronous LangGraph execution to FastAPI's asynchronous event loop.

### 4.1 Request Flow

- 1. POST /api/parse** — Input guardrail classification (Grog) + structured query parsing (Grog structured\_output). Returns ParsedQuery. On guardrail rejection: HTTP 403.
- 2. GET /confirm** — User reviews and edits parsed city, days, budget, interests. Human-in-the-loop checkpoint.
- 3. POST /api/plan** — Creates TripSession, fires \_run\_graph\_background() in ThreadPoolExecutor. Returns trip\_id immediately (non-blocking).
- 4. GET /api/plan/{id}/stream** — SSE endpoint. Polls session's threading.Queue, serialises events as data: {...}\n\n. Sends keep-alive comments every 15s. Closes on plan\_complete or plan\_error.
- 5. GET /brief/{trip\_id}** — brief.js fetches /api/plan/{id}/result and renders all five sections client-side.

## Section 5 — Sample Outputs

---

Three end-to-end test runs were executed to validate the complete pipeline. All five output sections were produced on every run without hitting the retry cap.

### Run 1 — Tokyo, 5 days, ¥80,000

Query: "Plan a 5-day trip to Tokyo in October, 80000 yen, temples and food"

Parsed: city=Tokyo, country=Japan, 5 days, ¥80,000 JPY, interests=[temples, food], month=October

```
Supervisor log: START → destination (first dispatch)
                 destination → weather (valid)
                 weather → budget (valid)
                 budget → itinerary (valid)
                 itinerary → packing (valid)
                 packing → END (all agents valid)
```

| Section     | Output Highlights                                                                                    |
|-------------|------------------------------------------------------------------------------------------------------|
| Destination | Japan 🇯🇵, Tokyo, JPY — Japanese yen, UTC+09:00, 2-paragraph editorial intro                          |
| Weather     | 7-day forecast, avg high ~27°C in May, no rain days                                                  |
| Budget      | ¥80,000 total → ¥16,000/day across 5 categories (accommodation, food, transport, activities, buffer) |
| Itinerary   | 5 days: Tsukiji market, Senso-ji, Shibuya crossing, Ghibli Museum, Meiji Shrine (4–5 segments/day)   |
| Packing     | 5 categories, 18 items total, warm-weather and temple-visit appropriate                              |

## Run 2 — Paris, 2 days, €400

Query: "Weekend trip to Paris, budget 400 euros, I love museums and pastries"

Parsed: city=Paris, country=France, 2 days, €400 EUR, interests=[museums, pastries]

| Section                 | Output Highlights                                                                                                |
|-------------------------|------------------------------------------------------------------------------------------------------------------|
| Budget                  | €400 / 2 days = €200/day — mid-range tier; accommodation 50% = €100/day, food + activities €100                  |
| Day 1 — Museum Marvels  | Louvre (9:00–11:30 AM, €18), Café Marly lunch (€25), Musée d'Orsay (2:00–4:30 PM, €12), Seine evening walk       |
| Day 2 — Pastry Delights | Ladurée Champs-Élysées (8–10 AM, €10), Montmartre neighbourhood (free), Boulangerie lunch, Sainte-Chapelle (€10) |

## Run 3 — Reykjavik, 7 days, \$3,000

Query: "7-day adventure trip to Reykjavik, 3000 USD budget, hiking and hot springs"

Parsed: city=Reykjavik, country=Iceland, 7 days, \$3,000 USD, interests=[hiking, hot springs]

| Section   | Output Highlights                                                                                                                                                                                          |
|-----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Weather   | Avg 9°C, 3 days with precipitation — packing validator correctly required rain gear                                                                                                                        |
| Budget    | \$3,000 → ~418,350 ISK at live rate → ~59,764 ISK/day (comfortable tier)                                                                                                                                   |
| Itinerary | Day themes: City Introduction, Nature Escape, Cultural Experience, Geothermal Adventure, Rainy Day Culture (indoor activities auto-scheduled for forecast rain days), Hiking Adventure, Geothermal Wonders |

| Section | Output Highlights                                                                                          |
|---------|------------------------------------------------------------------------------------------------------------|
| Packing | Rain gear added because forecast showed precipitation > 1mm on 3 days;<br>thermal layers added for avg 9°C |

## Section 6 — Issues Encountered and Solutions

| # | Issue                                                                 | Root Cause                                                                                                                                                                                                                                                                                                                      | Solution                                                                                                                                                                                                                                                                                                                        |
|---|-----------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | Starlette 1.0.0 Breaking Change — HTTP 500 on all page routes         | TemplateResponse API changed signature: old: <code>TemplateResponse("name", context_dict)</code> . New: <code>TemplateResponse(request, "name", context)</code> . With the old call, Starlette passed the context dict as the template name to Jinja2, which tried to hash it as a cache key and threw <code>TypeError</code> . | Updated all four page routes in <code>web/server.py</code> to the new (request, name, context) signature.                                                                                                                                                                                                                       |
| 2 | Groq <code>tool_use_failed</code> on itineraries longer than 4–5 days | Detailed itineraries with 5 segments × 4–6 rich fields each can exceed 4,000 output tokens. Groq's structured output (function-calling mode) has tighter output limits than the base completion endpoint and truncated the JSON mid-object.                                                                                     | Batched generation in <code>src/agents/itinerary.py</code> : trips are split into 4-day batches. Each batch is a separate LLM call returning an <code>Itinerary</code> with only those days. Batches are merged and sorted in Python. Keeps each call well under the limit and allows partial results to survive a batch retry. |
| 3 | LLM producing unlisted <code>DaySegment.period</code> values          | The period field was a strict Literal enum. The LLM occasionally generated "late morning" (not in the original list), causing Groq's schema validation to reject the response and return an empty itinerary.                                                                                                                    | Added "late morning" to the Literal type in <code>schemas.py</code> and to the prompt's period enumeration. Updated frontend <code>periodIcon()</code> to map the new value to a Material Symbols icon.                                                                                                                         |
| 4 | Windows cp1252 <code>UnicodeEncodeError</code> in CLI                 | The CLI renders using box-drawing characters (—, =), checkmarks (✓), and currency symbols (¥). Windows PowerShell defaults to cp1252 which cannot encode these characters.                                                                                                                                                      | Added <code>sys.stdout.reconfigure(encoding="utf-8", errors="replace")</code> at the top of <code>app.py</code> . This is a no-op on macOS and Linux (both default to UTF-8) and silently replaces unencodable characters on Windows rather than crashing.                                                                      |
| 5 | Budget validator false positives on rounded values                    | The original validator required the sum of category <code>daily_amounts</code> to equal <code>daily_budget_local</code> exactly. LLMs round integer amounts, so the sum of rounded values                                                                                                                                       | Validator now uses a 5% tolerance: <code>abs(daily_sum - daily_claimed) &gt; daily_claimed * 0.05</code> . This permits normal rounding while still catching genuinely wrong budget breakdowns (e.g. amounts                                                                                                                    |

| # | Issue | Root Cause                                              | Solution                                     |
|---|-------|---------------------------------------------------------|----------------------------------------------|
|   |       | rarely equals the authoritative daily budget precisely. | summing to 50% or 150% of the daily budget). |

## Section 7 — Lessons Learned

---

### 7.1 Deterministic Supervisor Was the Right Choice

Making the Supervisor a pure Python routing function with no LLM calls was the most impactful architectural decision. The routing logic is fast (microseconds vs. 1–2 seconds for an LLM call), predictable, and fully logged. Every routing decision appears in logs/supervisor.log with the agent name and reason. When an agent misbehaves, the exact issue and retry feedback are visible in the log, making root cause analysis straightforward.

An LLM-based supervisor would have been slower, more expensive per trip, and non-deterministic — it might route differently on identical inputs, making debugging unreliable. For a structured pipeline with known output shapes and a fixed agent order, deterministic validation is always the right choice over LLM-based routing.

### 7.2 Force-Scaled Budget Math

The Budget Agent uses the LLM only to determine proportional category splits (e.g. 40% accommodation, 25% food). All actual money amounts are computed in Python by multiplying the authoritative daily budget by the LLM-provided proportions and normalising. The LLM's raw number output is discarded.

This prevents the common failure mode where LLM-generated category amounts sum to 95% or 110% of the budget due to rounding errors. The math is always exact because Python, not the LLM, does the arithmetic.

### 7.3 Batched Itinerary Generation

Generating a 10-day itinerary in a single structured-output call would require over 8,000 output tokens, exceeding Groq's tool-call limits and producing truncated JSON. Splitting into 4-day batches keeps each call under 3,000 output tokens, well within limits, and also isolates failures: if batch 2 of 3 fails and retries, batches 1 and 3 are unaffected.

### 7.4 Travel Month vs Live Forecast for Packing

A user planning a December trip to Norway in May would get a warm May forecast if the live weather data drove the packing list — useless for packing winter gear. The packing agent uses `travel_month` as the primary climate signal (what December in Norway is typically like), and treats the live forecast as supplementary context only. The prompt explicitly explains this distinction to the LLM.

### 7.5 What Would Be Done Differently

- **Parallel agent execution:** Destination, Weather, and Budget agents are fully independent and could run in parallel using LangGraph's fan-out/fan-in pattern. Sequential execution costs 15–20 extra seconds per trip.
- **Streaming LLM output:** Groq supports token-level streaming. Streaming each 4-day itinerary batch in parallel would make the planning UI feel significantly more responsive.

- **Semantic output validation:** Current validators catch structural issues (wrong field types, missing fields, generic locations) but not semantic ones such as a restaurant that doesn't exist or an activity in the wrong district. A lightweight second-pass LLM check on the itinerary could catch these.
- **Persistent sessions:** The in-memory session store loses all trip data on server restart. A Redis or SQLite backend would make Trip Brief URLs shareable and permanent.



## Appendix — Assignment Requirements Compliance

### Must-Have Requirements

| Requirement                                             | Status  | Implementation                                                                                                               |
|---------------------------------------------------------|---------|------------------------------------------------------------------------------------------------------------------------------|
| LangGraph StateGraph (not AgentExecutor)                | ✓ DONE  | src/graph.py — full StateGraph with supervisor conditional edges                                                             |
| LLM: Google Gemini Flash 2.5                            | ADAPTED | Uses Groq Llama-3.3-70b. Same LangChain interface (.invoke, .with_structured_output). Swap is one line.                      |
| Destination Agent: real HTTP call to REST Countries     | ✓ DONE  | src/tools/rest_countries.py — live httpx GET, no mocking                                                                     |
| Weather Agent: real HTTP call to Open-Meteo             | ✓ DONE  | src/tools/open_meteo.py — two live calls: geocoding + forecast                                                               |
| Budget Agent: live FX rate + Pydantic structured output | ✓ DONE  | src/tools/exchange_rates.py + BudgetBreakdown in schemas.py                                                                  |
| All 5 output sections on every test query               | ✓ DONE  | Verified across 3 end-to-end runs (Tokyo, Paris, Reykjavik)                                                                  |
| AgentState as TypedDict with typed fields               | ✓ DONE  | src/state.py — fully typed TypedDict, factory function empty_state()                                                         |
| API keys in .env, .env never committed                  | ✓ DONE  | .env.example committed; .gitignore excludes .env                                                                             |
| Minimum 6 @tool functions                               | ✓ DONE  | 6 tools: fetch_country_info, geocode_city, fetch_weather_forecast, get_city_timezone, fetch_exchange_rates, convert_currency |
| Progress Report PDF                                     | ✓ DONE  | This document                                                                                                                |

## Good-to-Have (Bonus) Requirements

| Bonus Feature                                    | Status     | Notes                                                                   |
|--------------------------------------------------|------------|-------------------------------------------------------------------------|
| Human-in-the-loop checkpoint                     | ✓ DONE     | Confirm screen (web) and stdin prompt (CLI) before graph starts         |
| MemorySaver checkpoint                           | ✓ DONE     | Configured in graph.py, keyed by trip_id UUID                           |
| Web front-end (Streamlit suggestion)             | ✓ EXCEEDED | Full FastAPI + Vanilla JS app — 4 screens, SSE streaming, live progress |
| Supervisor validates each agent before advancing | ✓ DONE     | Full deterministic validation loop with retry + feedback injection      |
|                                                  |            |                                                                         |

## Key Divergences from Specification

**LLM Provider:** The assignment specified Google Gemini Flash 2.5 via ChatGoogleGenerativeAI. The project uses Groq (llama-3.3-70b-versatile). Both providers implement the identical LangChain chat interface — `.invoke()`, `.stream()`, `.with_structured_output(PydanticModel)` — so all architectural patterns are unchanged. The substitution was made for lower latency (Groq averages ~600 tokens/sec vs ~200 for Gemini Flash on the free tier) and a higher daily free-tier token limit.

**Supervisor Design:** The assignment specified an LLM-based supervisor ('None — LLM routing only'). The project uses a deterministic Python supervisor with per-agent output validators and a retry loop. This is a deliberate improvement over the specification: deterministic routing is faster, cheaper, fully auditable, and produces better output quality through the feedback-injection retry mechanism. The core architectural intent — a central coordinator that routes work to specialised agents — is fully satisfied.